

Aula 39: Deploy e Otimizações Finais - Teoria e Prática

Duração: 4 horas (2h teoria + 2h prática)

Parte Teórica (2h) - Preparando para Produção

1. Arquitetura do Projeto em Produção

- **Frontend:**
 - Build otimizado (Vue + Quasar).
 - Servido via Nginx (ou CDN).
- **Backend:**
 - Container Spring Boot com WebFlux.
 - Banco de dados PostgreSQL em container separado.

2. Docker e Docker Compose

- **Vantagens:**
 - Isolamento de ambientes.
 - Facilidade de deploy e escalabilidade.
- **Componentes:**

Diagram

Code

```
graph LR
  A[Frontend] --> B[Nginx]
  B --> C[Internet]
  D[Backend] --> E[PostgreSQL]
  C --> D
```

3. Otimizações para Tempo Real

- **Backend:**
 - Configurar `WebSocketBufferSize` no Spring.
 - Usar `@Scheduled` para limpar sessões inativas.
- **Frontend:**
 - Debounce em eventos de digitação.
 - Virtualização de lista para muitas mensagens.

4. Segurança Básica

- **WebSockets:**
 - Adicionar CORS restrito.
 - Autenticação via JWT (STOMP headers).
- **SSE:**
 - Validar origem das requisições.

Parte Prática (2h) - Deploy com Docker

1. Dockerizando o Backend (1h)

Arquivo: backend/Dockerfile

```
dockerfile

FROM eclipse-temurin:17-jdk-jammy
WORKDIR /app
COPY build/libs/reactive-chat-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Comandos para build:

```
bash

./gradlew build
docker build -t chat-backend .
```

2. Dockerizando o Frontend (1h)

Arquivo: frontend/Dockerfile

```
dockerfile

FROM node:18 as builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
```

```
COPY --from=builder /app/dist/spa /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Arquivo: frontend/nginx.conf

```
nginx

server {
    listen 80;
    location / {
        try_files $uri $uri/ /index.html;
    }
    location /ws-chat {
        proxy_pass http://backend:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "Upgrade";
    }
}
```

3. Docker Compose (Integração Final)

Arquivo: docker-compose.yml

```
yaml

version: '3.8'

services:
  postgres:
    image: postgres:14
    environment:
      POSTGRES_PASSWORD: chat123
      POSTGRES_DB: chatdb
    volumes:
      - pgdata:/var/lib/postgresql/data

  backend:
    build: ./backend
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/chatdb
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: chat123
    depends_on:
      - postgres
```

```
frontend:
  build: ./frontend
  ports:
    - "80:80"
  depends_on:
    - backend

volumes:
  pgdata:
```

Comando para iniciar:

```
bash
```

```
docker-compose up --build
```

Atividade Prática

Desafio: Implemente estas melhorias:

1. Variáveis de Ambiente:

- Modifique o `docker-compose.yml` para usar um arquivo `.env` com senhas.

2. Health Checks:

- Adicione verificações de saúde para o PostgreSQL e backend.

Solução:

```
yaml
```

```
# docker-compose.yml
services:
  backend:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 3

  postgres:
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
```

```
bash
```

```
# .env
POSTGRES_PASSWORD=supersenha
SPRING_DATASOURCE_PASSWORD=supersenha
```

Exercício de Otimização

Problema: O chat está lento com 1000+ mensagens.

Tarefas:

1. Implemente paginação no backend:

```
java

@GetMapping("/messages")
public Flux<Message> getMessages(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "50") int size
) {
    return messageRepository.findAll()
        .skip(page * size)
        .take(size);
}
```

2. No frontend, use `q-infinite-scroll` para carregar mensagens antigas sob demanda.

Recursos Adicionais

- Docker Best Practices: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
- Spring Boot Docker: <https://spring.io/guides/gs/spring-boot-docker/>

Projeto Concluído! 🎉

Próximos Passos: Monitoramento com Prometheus + Grafana (opcional).